

Technical Journey in AI/ML — MLSS Application

Kanishka Shukla

My journey into AI and machine learning started not with a textbook but with a question: what happens when a machine learns? That curiosity drove me to build, break, and rebuild systems until I felt I had a clear answer. My first organized step was an ML internship at CodeAlpha from May to June 2025. There, I moved beyond theory into real-world workflows. I worked on feature engineering, cross-validation, and hyperparameter tuning. I also learned how to explain model performance to both engineers and non-technical stakeholders. That experience grounded my intuition for the full ML lifecycle: data ingestion, preprocessing, training, evaluation, and documentation within agile sprints.

From there I pushed into applied LLM engineering. Trinethra, my AI Feedback Analyzer, runs supervisor call transcripts through a locally hosted Llama 3.2 pipeline and returns structured JSON — sentiment evidence, rubric scores, KPI mapping, and follow-up questions — in a single inference pass. The core challenge was structured output reliability: I built JSON enforcement, extraction layers, and parsing fallback handlers, discovering firsthand how brittle naive prompting is and how much engineering sits beneath a "working" LLM feature. My most ambitious project is a Document Intelligence and Agentic RAG system I built as an AI Engineer Intern Assessment. The stack handles scanned PDFs, handwritten pages, image-heavy reports, and tables using pdfplumber, pdf2image, and pytesseract for OCR. Each page produces extracted text and a rendered image — both stored. An LLM classifier then tags each document across multiple dimensions: type, topic, sensitivity level, and content characteristics, outputting structured JSON with a self-designed schema. On top of this sits an agentic RAG pipeline that retrieves relevant chunks and synthesises grounded answers with inline citations (document name + page number), refusing to hallucinate when no relevant content exists. The chatbot renders cited page thumbnails inline; clicking opens the full page. A separate bulk-upload page shows per-file processing status (parsing → classifying → indexed). I also implemented a full security model across every layer — upload validation, encrypted storage, sandboxed processing, and rate-limited retrieval — and documented the threat model, decisions made, and trade-offs in the README.

Alongside this, my Food AI Safety System uses a Java/Spring Boot agentic pipeline routing natural-language food queries through multi-stage ML classification to structured safety assessments. And GigFlow, my production CRM, gave me deep experience with Docker, CI/CD, JWT auth, and RBAC.

What MLSS will give me that I cannot self-teach is rigorous mathematical depth — the statistical learning theory, probabilistic graphical models, and optimisation foundations that explain why these systems behave as they do. I can build a RAG

pipeline; I want to understand when retrieval-augmented generation probably outperforms fine-tuning, and under what distributional assumptions. I seek the gap between practitioner and researcher, and MLSS is precisely that bridge.